

Anàlisi i Recomanacions per a la Implementació d'un Algorisme de 3D Bin Packing en Python

Aquest informe ofereix una anàlisi exhaustiva del problema d'empaquetament 3D bin packing, dirigida a l'autor de la consulta que busca millorar una aplicació existent desenvolupada en Python. L'anàlisi es centra en les biblioteques disponibles, algoritmes eficaços, estratègies de detecció de col·lisions i tècniques avançades per gestionar objectes complexos i irregulars. Es proporcionen recomanacions concretes i exemples de codi basats exclusivament en la informació disponible per ajudar a implementar una solució robusta, precisa i automàtica.

Evaluació d'Eines i Biblioteques Python per a 3D Bin Packing

La selecció de la llibreria adequada és un punt de partida crític per a qualsevol projecte d'empaquetament 3D. Diverses eines són presents en el context proporcionat, cada una amb avantatges i desavantatges específics. La decisió final depèn de si es prefereix una integració total en Python, l'accés a funcionalitats especialitzades mitjançant API o una combinació d'ambdues.

Py3dbp⁹ és probablement la llibreria més accessible i directa per a una implementació ràpida. Aquesta eina, escrita completament en Python i llicenciada MIT, implementa un algorisme basat en una heurística com **First Fit Decreasing** adaptada per a tres dimensions^{9 19}. Ofereix funcionalitat clau com l'ordenació d'objectes (per exemple, **bigger_first**), distribució d'elements (**distribute_items**) i precisió decimal configurable (**number_of_decimals**). El seu ús principal consisteix a afegir **Bin** objects (els contenidors) i **Item** objects (els objectes a empaquetar) i executar la funció d'empaquetament. Aquesta llibreria es pot instal·lar fàcilment mitjançant **pip install py3dbp** i està dissenyada per gestionar múltiples contenidors, retornant els objectes que s'han ajustat correctament i aquells que no han pogut ser col·locats⁹. Donada la naturalesa "simple i robusta" que es demana, py3dbp representa una opció molt viable per a una primera iteració.

Per a requisits més avançats, cal considerar biblioteques més poderoses però potser menys immediates. La llibreria trimesh¹⁴, ja utilitzada en l'aplicació actual, va més enllà de la càrrega de models. Incorpora una classe **CollisionManager** que permet gestionar col·lisions entre malles 3D i inclou mètodes com **in_collision_internal** i **in_collision_single** per detectar interseccions¹⁴. Una característica particularment útil és la funció **mesh_to_BVH**, que crea una jerarquia de volums envoltants (Bounding Volume Hierarchy, BVH) a partir d'un objecte Trimesh¹⁴. Aquesta estructura accelerada és essencial per a realitzar milions de comprovacions de col·lisió en temps acceptable, resolent així un dels punts febles identificats en l'algorisme actual. A més, trimesh pot generar dades detallades de contacte, com la profunditat de penetració i la normal de la superfície, que poden ser útils per a simulacions més sofisticades¹⁴.

L'enfocament programari també inclou alternatives externes i híbrides. Un recurs interessant és l'API comercial 3DBinPacking.com^{8 28}. Aquest servei web accepta sol·licituds JSON per empaquetar

ítems cúbics o rectangulars en diversos contenidors. Permet especificar rotacions (**vr:true**), definir modes d'optimització com la minimització del nombre de caixes i retorna les coordenades d'emplaçament, el percentatge d'espai utilitzat i fins i tot imatges SVG del resultat ⁸. Encara que aquesta opció transferiria part de la lògica de l'algorisme fora de l'aplicació, podria ser una solució vàlida si es vol evitar la complexitat de construir i mantenir un motor d'empaquetament propi, especialment donat el límit de 4999 ítems per simulació ²⁸.

Finalment, el camp de l'aprenentatge per reforç (Reinforcement Learning, RL) ha generat eines innovadores com DeepPack3D ²¹. Aquest paquet de Python, compatible amb Python 3.10 i TensorFlow 2.10.0, ofereix diverses heurístiques com Best Lookahead (BL), Best Area Fit (BAF) i BLSF, juntament amb un agent entrenat amb aprenentatge per reforç ²¹. Pot llegir dades d'entrada manualment o des de fitxers, oferint visualització i suport a GPU ²¹. Si bé l'objectiu inicial era un algoritme simple, DeepPack3D demostra el potencial d'aquestes tècniques per trobar seqüències d'empaquetament òptimes, encara que sovint amb restriccions addicionals com la no rotació dels elements ^{18 19}. La seva integració podria representar una capa de personalització futura.

Biblioteca/API	Desenvolupament	Llicència	Característiques Clau	Complexitat Inicial
py3dbp	Python	MIT	Heurística First Fit Decreasing, gestió de múltiples contenidors, ordenació i distribució d'elements ⁹ .	Baixa
trimesh	Python	MIT/PSF	Gestió de col·lisions (CollisionManager), creació de BVH, càrrega i processament de malles ¹⁴ .	Mitjana
3DBinPacking.com API	Servei web (Python client)	Comercial	Optimització per volum o nombre de caixes, rotació, sortida detallada, visualització ^{8 28} .	Molt baixa (integració)
DeepPack3D	Python	MIT	Heurístiques i agents d'aprenentatge per reforç, suport a GPU, visualització ²¹ .	Alta
HPP-FCL / FCL	C++ / Python	BSD-3-Clause	Biblioteca de detecció de col·lisions generalista, suporta molts tipus de formes ³⁸ .	Mitjana a Alta

En resum, per a una implementació ràpida i robusta, es recomana començar amb **py3dbp**. Aquesta llibreria aborda directament el problema d'empaquetament i té una interfície senzilla. Per a una integració més profunda i control sobre la detecció de col·lisions, **trimesh** ofereix les eines necessàries per crear una solució més personalitzada i precisa, especialment a través de la seva capacitat de crear BVH.

Estratègies Avançades de Detecció de Col·lisions per a Models 3D Complexos

Un dels problemes més crítics identificats en l'algoritme actual és la fallida en la detecció de col·lisions, tant entre peces com amb les parets del contenidor. L'actual estratègia de graella rígida només funciona per a caixes idèntiques i ortogonals, i fracassa completament amb formes irregulars o quan s'intenta provar diferents orientacions. Una solució efectiva requereix una estratègia de detecció de col·lisions en dues fases: una fase ampla (broad phase) per descartar ràpidament parelles d'objectes que no poden estar en col·lisió, i una fase estreta (narrow phase) per a una comprovació precisa entre parelles candidates.

La fase ampla es beneficia enormement de les jerarquies de volums envoltants (Bounding Volume Hierarchies, BVH). En comptes de comparar cada objecte amb tots els altres (una operació $O(n^2)$), una BVH organitza els objectes en una estructura arborescent. Cada node intern de l'arbre representa una caixa envoltant (bounding box) que conté les caixes dels seus fills. Això permet descartar grans agrupacions d'objectes de cop si les seves caixes envoltant principals no col·lideixen. Dins del context proporcionat, **trimesh** ofereix directament la funcionalitat per crear una BVH a partir d'una mesh mitjançant la funció `mesh_to_BVH`¹⁴. Aquesta funció genera una estructura jeràrquica eficient que accelera dràsticament la detecció de col·lisions. Altres fonts confirmen que les BVH són una tècnica estàndard per accelerar operacions espacials en geometria 3D^{23 34}. Tot i que JavaScript, la biblioteca **three-mesh-bvh** també serveix com a excel·lent referència per entendre com funcionen aquestes estructures, que divideixen l'espai de manera eficient per a consultes ràpides^{15 16}.

Una alternativa a la BVH són les octrees, una forma de subdivisió espacial que divideix recursivament l'espai en vuit quadrants o cel·les^{22 25}. Són útils per gestionar escenes massives i accelerar operacions com el raycasting i la detecció de col·lisions²⁷. Tanmateix, una font suggereix que una octree desplegada sobre geometria 3D pot convergir cap a una estructura similar a una BVH, ja que al final cada primitiva (triangle) encara necessita la seva pròpia caixa envoltant²⁵. Per tant, una BVH construïda sobre les primitives de la mesh (els triangles) és sovint més directa i eficient per a la detecció de col·lisió de models 3D complexos.

La fase estreta s'activa només pels parells d'objectes que passen la prova de la fase ampla. Aquí, es realitza una comprovació geomètrica precisa. Per a models 3D carregats des de STL o OBJ, això implica fer proves d'intersecció triangle-triangle⁶. **trimesh** gestiona internament aquest procés quan es fa servir el seu **CollisionManager**¹⁴. Una altra opció més genèrica, especialment útil si es vol externalitzar la detecció de col·lisions, és la llibreria HPP-FCL (Humanoid Path Planner - Collision Detection Library)³⁸. Escrita en C++ però amb una interfície Python, aquesta biblioteca ofereix una àmplia gamma de funcions de detecció de col·lisions, distància i marges de seguretat per a molts tipus de formes, incloses malles, caixes, esferes i cilindres³⁸. La seva flexibilitat la

converteix en una elecció potent per a projectes que requereixen una gran robustesa o que treballen amb una varietat de tipus de sensors o models físics.

Per a una implementació en Python, la combinació de **trimesh** per a la manipulació de la geometria i la creació de la BVH, juntament amb el seu **CollisionManager** per a la fase estreta, ofereix una solució completa i eficient. Aquesta aproximació evita la dependència de biblioteques externes i s'integra perfectament amb l'ecosistema actual de l'aplicació. La llista d'objectes 3D (**myMeshes**) es pot mantenir com a estat de l'aplicació, on cada vegada que una peça es mou, es pot fer una comprovació de col·lisió contra totes les altres peces i les parets del contenidor utilitzant el **CollisionManager** ²⁴. Aquest enfocament modular permet tractar el problema de la detecció de col·lisions com una capa separada i reutilitzable del sistema d'empaquetament.

Implementació d'Algoritmes Heurístics per a Empaquetament 3D Automàtic

Amb una solució robusta per a la detecció de col·lisions, el següent pas és implementar l'algoritme d'empaquetament en si. L'exigència de "tot automàtic" implica que l'algorisme ha de ser capaç de gestionar sistemàticament la rotació dels objectes per trobar la millor orientació possible sense intervenció humana. L'algoritme actual intenta això, però de forma massa simplista, provant només una llista fixa de rotacions sense provar-les realment ¹.

Una heurística bàsica i força eficaç per al problema d'empaquetament 3D (3D-BPP) és l'estratègia **Bottom-Left-Fill** (BLF) o una de les seves variants ¹³. Aquest tipus d'algoritme pren els objectes i els col·loca tan avall i tan a l'esquerra com sigui possible dins de l'espai disponible. Una versió més sofisticada, com la implementada en un article recollit en el context, utilitza una estratègia BLF que opera sobre "punts pivot" per col·locar objectes irregulars, garantint que no flotin i considerant la compressió dinàmica per augmentar l'eficiència ¹³. Aquest algorisme pot ser adaptat per a provar les sis orientacions ortogonals possibles per a cada objecte (rotacions de 90° al voltant dels tres eixos) ^{10 13}.

Una altra família d'algoritmes populars són els d'empaquetament per nivells o capes (**Layer-based packing**). Aquests algoritmes construeixen primer capes compactes d'objectes idèntics i després empilen aquestes capes dins el contenidor ^{10 12}. Una heurística de tres etapes proposada per Harrath (2022) segueix aquest paradigma: genera capes, crea candidats de solució a partir d'aquestes capes i finalment col·loca les capes en el contenidor segons el candidat que maximitzi l'ús del volum ¹⁰. Aquest enfoig sistemàtic de crear subestructures (capes) abans de posar-les conjuntament pot portar a solucions més estables i eficients que un enfoig de "posa-un-objecte-qualsevol".

El treball de Wang i col·laboradors presenta un algorisme híbrid genètic (HGA) que combina l'algoritme genètic (GA) amb la cerca tabú per optimitzar l'ordre d'empaquetament i l'estat de rotació dels ítems ². Utilitzen una codificació de dues etapes basada en claus aleatòries per determinar l'orientació (amb 6 estats de rotació possibles per a càrregues prismàtiques) i un algorisme heurístic de càrrega (COHLA) basat en la divisió i fusió de l'espai residual per realitzar el posicionament ². Aquesta és una aproximació extremadament potent, especialment per a problemes amb una gran quantitat d'objectes i restriccions. No obstant això, la seva complexitat pot ser excessiva per a una implementació inicial, encara que el seu principi —provar sistemàticament combinacions d'ordre i

rotació— és molt informatiu. El model de programació lineal mixta entera (MILP) també pot resoldre el problema exactament, introduint variables binàries per a cada ítem i orientació i usant tècniques com la generació de columnes per a optimitzar ⁴.

Per a una implementació pràctica i escalable, es pot seguir un enfoig híbrid: 1. Pre-procesament: Per a cada objecte, calculi's totes les seves orientacions possibles (com a màxim 6). Per a cada orientació, es pot calcular una caixa envoltent ortogonal (AABB) i registrar-la. 2. Selecció de l'objecte: En cada pas de l'algoritme, seleccioni's l'objecte encara no empaquetat que tingui el millor "valor" segons alguna heurística. Aquest valor pot basar-se en l'àrea de la base (per tal de crear una base estable), el volum o una combinació d'ambdós. 3. Selecció de la posició: Iteri's sobre totes les orientacions possibles de l'objecte seleccionat. Per a cada combinació (objecte + orientació), es busca la millor posició possible dins del contenidor utilitzant una regla de posicionament com **Bottom-Left-Front**. 4.

Comprovació de col·lisió: Abans de col·locar definitivament l'objecte, es fa una comprovació de col·lisió utilitzant la **CollisionManager** de **trimesh** per assegurar-se que no travessa les parets ni altre material empaquetat. 5. Aplicació de la millor opció: Si s'ha trobat una posició vàlida, es col·loca l'objecte i es repeteix el procés fins que no hi hagi més objectes per empaquetar.

Aquest enfoig sistemàtic, que integra la detecció de col·lisions en cada pas de selecció, superarà de llarg la lògica actual de graella rígida. L'article que descriu un algorisme BLF per a objectes irregulars ja ofereix pseudo-codi que pot guiar l'implementació d'aquesta fase de posicionament ¹³. Aquest enfocament, encara que més complex que el que actualment es disposa, és fonamental per aconseguir una eficiència d'empaquetament raonable, especialment amb formes irregulars.

Gestionant Rotacions i Formes Irregulars per Maximitzar l'Eficiència

Un dels aspectes més complicats de l'empaquetament 3D és la gestió de les rotacions de les peces i la seva adaptació a formes irregulars. L'algoritme actual falla completament en aquest aspecte, col·locant peces de forma ineficient i sense provar cap rotació significativa ¹. Per aconseguir una alta eficiència, especialment amb formes complexes que no són caixes regulars, cal adoptar una estratègia que integri la detecció de col·lisions 3D i l'anàlisi de la geometria de cada peça individualment.

L'enfocament més directe per gestionar les rotacions és provar-les sistemàticament. Per a objectes que permeten rotacions ortogonals (rotacions de 90° al voltant dels seus eixos locals), cada objecte pot tenir fins a sis orientacions úniques ^{10 13}. L'algoritme híbrid genètic (HGA) mencionat anteriorment utilitza explícitament aquestes sis orientacions per a caixes prismàtiques ². La implementació consisteix a, per a cada objecte que s'ha de col·locar, generar totes les seves versions rotades i triar la que resulti en la millor ubicació. Aquesta idea es pot integrar en qualsevol dels algorismes heurístics discutits. Per exemple, en una heurística **Best Fit**, en lloc de calcular simplement la millor posició per a una orientació fixa, es calcularia la millor posició i orientació combinada.

Quan es treballa amb formes irregulars, com un Z-shape o un triangle rectangle, el concepte de "millor posició" esdevé més complex. Aquí, la detecció de col·lisions 3D precisada amb una jerarquia de volums envoltants (BVH) es converteix en indispensable ^{6 14}. L'algoritme no pot

confiar en simples comparacions d'extrems; ha de comprovar si dues formes 3D arbitràries es superposen. L'article que descriu un algorisme per a objectes irregulars i deformables utilitza una estratègia **Bottom-Left-Fill** que col·loca objectes utilitzant punts pivot i considera la compressió dinàmica sense deixar buits¹³. Aquesta és una indicació clara que per a formes irregulars, l'algorisme ha de ser capaç de "ajustar-se" a l'espai disponible.

Per gestionar sistemàticament aquesta adaptabilitat, es pot implementar un sistema de "fitness" o qualitat per a cada possible colocació. Aquest sistema podria considerar diversos factors: * Estabilitat: Quantes cares de l'objecte estan en contacte amb el fons del contenidor o amb un altre objecte?² * Contacte lateral: Hi ha contacte amb les parets laterals del contenidor o amb altres objectes? Això pot ajudar a subjectar l'objecte i evitar moviments posteriors. * Minimització de buits: Quina és l'àrea de la cara inferior de l'objecte que està en contacte amb l'espai empaquetat? Una major àrea de contacte generalment significa menys buits. * Alineació amb eixos locals: Pot ser desitjable alinear certs objectes amb els eixos del contenidor principal per a una organització posterior.

La implementació d'aquest sistema de fitness requereix una bona integració entre l'algorisme d'empaquetament i el sistema de detecció de col·lisions. En cada iteració, es podria provar una possible colocació, fer una comprovació de col·lisió temporal, i si no hi ha col·lisió, calcular el seu "score". Finalment, es seleccionaria la colocació amb el puntatge més alt.

Per a objectes amb un nombre reduït de cares, com un prisma triangular, es pot considerar una optimització addicional. En comptes de rotar sempre amb transformacions 3D complexes, es podrien pre-calcular les seves caixes envoltent ortogonals (AABBs) per a cada una de les seves orientacions òptimes. Això podria accelerar el procés de detecció de col·lisions en la fase amplia, ja que les AABBs són molt més ràpides de comprovar que les geometries 3D complexes. Aquesta idea de distingir entre objectes simples (que poden usar AABB) i complexos (que necessiten OBB/BVH) es correspon exactament amb el que es demana a la consulta original¹.

Integració del Model d'Empaquetament en l'Aplicació Python Existent

Integrar un nou motor d'empaquetament robust en l'aplicació existent exigeix una planificació estratègica per substituir gradualment la lògica deficiente per una nova i més precisa. El codi actual, malgrat ser defectuós, probablement conté parts vàlides com la interfície d'usuari (**tkinter**), la càrrega de models (**trimesh**) i la visualització (**PyVista**). El nucli central, **_fallback_optimization**, és el que cal substituir.

La primera fase consisteix en netejar i preparar el terreny. Cal identificar i documentar quina part del codi actual funciona i quina no. Les funcions com **_simple_grid_packing** són un obstacle clar que cal eliminar. L'estructura general de l'aplicació, que probablement involucra una llista d'objectes a empaquetar i un objecte que defineix el contenidor, pot mantenir-se. El canvi resideix en el mecanisme que assigna les posicions a cada objecte.

La segona fase és l'implementació del nucli d'empaquetament. Basant-nos en les recomanacions anteriors, es proposa un nou mòdul o classe anomenada **OptimizedPackingSystem**. Aquest

sistema tindrà una interfície clara, probablement una funció `pack(contenedor, objectes)` que tornarà una llista de posicions transformades (o llistes de resultats).

A continuació es presenta un esquema de codi conceptual per a aquest nou sistema:

```
## Conceptual code for the new optimized packing system
import trimesh
import numpy as np
from some_module import PositionAndRotation # Custom class to store position and rotation

class OptimizedPackingSystem:
    def __init__(self):
        # Initialize a collision manager for efficient collision detection
        self.collision_manager = trimesh.collision.CollisionManager()

    def _get_all_orientations(self, mesh):
        """
        Generate all 6 orthogonal orientations of a mesh.
        This is a simplified example; a full implementation would handle more complex rotations.
        """
        orientations = []

        # The identity orientation (no rotation)
        orientations.append((np.eye(3), np.array([0, 0, 0])))

        # Add more rotations here...
        # For simplicity, this is just a placeholder for the logic that would generate other rotations.
        # Each rotation would create a new mesh or just store the rotation matrix and origin.

        return orientations

    def _find_best_placement(self, piece_mesh, container_bounds, placed_meshes):
        """
        Find the best position and orientation for a single piece.
        """
        self.collision_manager.clear()
        for placed_mesh_data in placed_meshes:
            # Add each already placed mesh to the collision manager with its position and rotation
            self.collision_manager.add_object(id(len(self.collision_manager)), piece_mesh,
                                              geometry=placed_mesh_data['geometry'],
                                              transform=placed_mesh_data['transform'])

        best_score = -1
        best_placement = None

        # Get all possible orientations for this piece
        all_orientations = self._get_all_orientations(piece_mesh)

        for rotation_matrix, rotation_origin in all_orientations:
            # Transform the piece based on the current orientation
            rotated_mesh = piece_mesh.copy()
            # Apply rotation... (this is a placeholder for the actual transformation logic)
```

```

        # For each possible orientation, try to find a valid placement
        # This involves iterating over potential positions and checking
        # This is the core of the algorithm where we'd implement the logic

        # This is a highly simplified check
        test_position = np.array([0, 0, 0]) # This should be calculated
        test_transform = trimesh.transformations.translation_matrix(test_position)

        # Temporarily place the mesh to check for collisions
        # Check for collision with container walls first...
        # Then check for collision with other placed meshes
        if not self._is_in_container(rotated_mesh, container_bounds, test_transform):
            # Not in container, skip
            continue

        # Calculate a "fitness" score for this placement
        score = self._calculate_placement_score(rotated_mesh, test_transform)

        if score > best_score:
            # In a real implementation, we'd also get the rotation
            best_score = score
            # Store the transform instead of just position
            best_placement = {
                'position': test_position,
                # 'rotation': rotation_matrix, # Store the rotation
                'transform': test_transform
            }

    return best_placement

def _is_in_container(self, mesh, container_bounds, transform):
    """
    Check if a transformed mesh is completely inside the container bounds.
    """
    # Apply the transform to the mesh bounds and check against container bounds
    # Implementation left as an exercise...
    pass

def _calculate_placement_score(self, mesh, transform, placed_meshes):
    """
    Calculate a score for how good a placement is.
    """
    # Implement a scoring system based on stability, contact area, etc.
    # This is a placeholder.
    return 1

def pack(self, container_dimensions, list_of_piece_dicts):
    """
    Main public method to perform the packing.
    :param container_dimensions: [length, width, height]
    :param list_of_piece_dicts: List of dicts containing 'mesh' and other attributes
    :return: List of dictionaries with 'transform' for each piece that was successfully packed
    """

```



```

"""
container_bounds = [np.array([0, 0, 0]), np.array(container_dimensions)]
successfully_packed = []
placed_objects = []

while list_of_piece_dicts:
    # For simplicity, let's select the next piece to place
    # In a real implementation, we'd use a heuristic to choose the next piece
    piece_data = list_of_piece_dicts.pop(0)
    piece_mesh = piece_data['mesh']

    # Find the best placement for this piece
    placement = self._find_best_placement(piece_mesh, container_bounds)

    if placement:
        # If a valid placement was found, add it to the list of placed objects
        # We store the original mesh and the final transform
        placed_objects.append({
            'mesh': piece_mesh,
            'transform': placement['transform']
        })
        successfully_packed.append({
            'id': piece_data['id'], # Assuming each piece has a unique id
            'transform': placement['transform']
        })
    else:
        # If no placement was found, we couldn't pack this piece
        print(f"Could not pack piece {piece_data['id']}")

return successfully_packed

```

Aquest pseudocodi outline una estructura modular. El **OptimizedPackingSystem** gestiona la detecció de col·lisions i encapsula la complexitat de la lògica d'empaquetament. La funció **pack** és l'interfície pública que l'aplicació principal cridarà. Aquesta funció iterativament selecciona peces, troba la millor ubicació possible per a elles i les afegeix a l'espai empaquetat. Aquesta aproximació incremental permet reemplaçar l'algorisme antic bloc a bloc, mantenint l'estructura general de l'aplicació intacta durant el procés de refactorització.

Recomanacions Finales i Resum de Passos per a una Solució Robusta

Després d'una anàlisi exhaustiva del problema d'empaquetament 3D i de les solucions disponibles, es poden sintetitzar una sèrie de recomanacions pràctiques per transformar l'aplicació actual en una eina robusta, precisa i automàtica. L'objectiu final és superar les limitacions de l'algorisme actual, que falla en la detecció de col·lisions i en la gestió de la rotació d'objectes irregulars.

La primera i més crítica recomanació és substituir l'algorisme actual. La lògica de la graella rígida (**_simple_grid_packing**) ¹ és inherentment inadequada per a formes 3D complexos i no té en compte les col·lisions. La seva eliminació és un pas indispensable.

La segona recomanació és prioritzar la detecció de col·lisions precisa. Aquesta és la pedra angular de qualsevol algoritme d'empaquetament efectiu. Es recomana utilitzar la biblioteca **trimesh** per a aquesta tasca ¹⁴. Es deu crear un **CollisionManager** ¹⁴ i utilitzar-lo per a totes les comprovacions de col·lisió. Per accelerar el procés, especialment en escenes amb moltes peces, es recomana convertir cada mesh empaquetada en una jerarquia de volums envoltants (BVH) mitjançant **mesh_to_BVH** ¹⁴. Aquesta doble capa de detecció (BVH per a la fase ampla i geometria per a la fase estreta) és una pràctica estàndard i molt eficient ^{6 23}.

La tercera recomanació és implementar una heurística de posicionament avançada. En lloc d'un mètode simple, es proposa implementar un algoritme de tipus **Bottom-Left-Fill** (BLF) o una variant basada en capes ^{10 13}. Aquest algoritme ha de ser modificat per a provar sistemàticament totes les orientacions ortogonals (fins a 6) de cada objecte abans de decidir on col·locar-lo ^{2 13}. Aquesta exploració sistemàtica de l'espai de solucions (orientacions i posicions) és el que permet aconseguir una eficiència d'empaquetament superior, especialment per a formes irregulars.

La quarta recomanació és considerar biblioteques de tercers parts per una implementació més ràpida. Si l'objectiu prioritari és la funcionalitat i la rapidesa, es podria començar amb **py3dbp** ⁹. Aquesta llibreria implementa una heurística robusta i pot ser una solució satisfactòria per a molts casos d'ús. Només si es necessiten més control o funcionalitats especialitzades (com la gestió de la rotació per a models 3D complets) caldrà passar a una implementació més personalitzada utilitzant **trimesh**.

Finalment, la cinquena recomanació és planificar la migració de forma modular. En lloc de reescriure tot el programa d'un cop, es proposa crear un nou mòdul (**OptimizedPackingSystem**) que contingui tota la nova lògica d'empaquetament. Aquest mòdul actuarà com un "motor" que l'aplicació principal podrà cridar amb una interfície clara (**pack(contenedor, objectes)**) ³⁷. Aquest enfoig modular facilitarà la prova, la depuració i la integració progressiva de la nova funcionalitat, protegint alhora les parts de l'aplicació que ja funcionen correctament (com la interfície i la càrrega de models).

En resum, per aconseguir una solució que sigui "simple i robusta", "detecti col·lisions correctament" i "maximitzi l'eficiència d'espai", cal adoptar un enfoig en quatre passes: 1. Substituir l'algoritme de posicionament actual. 2. Construir un sistema de detecció de col·lisions basat en **trimesh** i BVH. 3. Implementar una heurística de posicionament (com BLF) que provarà sistemàticament les rotacions. 4. Integrar aquest nou sistema de manera modular dins de l'aplicació existent.

Seguint aquest pla d'acció, es pot transformar l'aplicació actual en una eina d'empaquetament 3D d'alt rendiment que compleixi plenament les necessitats de l'usuari.

en

1. [PDF] Algorithms for 3-D Geometric Bin Packing https://hg.gatech.edu/sites/default/files/attachments/arindamkhan_arcpropf12pallet.pdf

2. A 3D Offline Packing Algorithm considering Cargo Orientation and ... <https://onlinelibrary.wiley.com/doi/10.1155/2023/5299891>
3. The 3D bin packing problem for multiple boxes and irregular items ... <https://link.springer.com/article/10.1007/s10489-023-04604-6>
4. A column generation-based heuristic for the three-dimensional bin ... https://www.researchgate.net/publication/314657085_A_column_generation-based_heuristic_for_the_three-dimensional_bin_packing_problem_with_rotation
5. On-line three-dimensional packing problems: A review of off-line ... <https://www.sciencedirect.com/science/article/abs/pii/S0360835222001929>
6. 3D Collision Detection Library for Python and C++ - MeshLib <https://meshlib.io/feature/collision-detection/>
7. 2D/3D collision detection library for Python — pycollision 0.0.2 ... <https://pycollision.readthedocs.io/en/latest/>
8. Introduction – API Reference - 3DBinPacking <https://www.3dbinpacking.com/en/api-doc>
9. enzoruiz/3dbinpacking: A python library for 3D Bin Packing - GitHub <https://github.com/enzoruiz/3dbinpacking>
10. A three-stage layer-based heuristic to solve the 3D bin-packing ... <https://www.sciencedirect.com/science/article/pii/S1319157821001749>
11. Solving the three-dimensional open-dimension rectangular packing ... <https://www.sciencedirect.com/science/article/abs/pii/S0305054824001230>
12. A layer-building algorithm for the three-dimensional multiple bin ... <https://www.sciencedirect.com/science/article/pii/S2405896315003687>
13. [PDF] A Constructive Heuristic Algorithm for 3D Bin Packing of Irregular ... <https://arxiv.org/pdf/2206.15116>
14. trimesh.collision - trimesh 4.7.4 documentation <https://trimesh.org/trimesh.collision.html>
15. gkjohnson/three-mesh-bvh - GitHub <https://github.com/gkjohnson/three-mesh-bvh>
16. Three-mesh-bvh: A plugin for fast geometry raycasting and spatial ... <https://discourse.threejs.org/t/three-mesh-bvh-a-plugin-for-fast-geometry-raycasting-and-spatial-queries/26394>
17. Mochi: Fast & Exact Collision Detection - arXiv <https://arxiv.org/html/2402.14801v4>
18. Python example of 3D bin packing problem and visualization <https://stackoverflow.com/questions/68953770/python-example-of-3d-bin-packing-problem-and-visualization>
19. luisgarciar/3D-bin-packing - GitHub <https://github.com/luisgarciar/3D-bin-packing>
20. trimesh.path.packing - trimesh 4.7.4 documentation <https://trimesh.org/trimesh.path.packing.html>

21. DeepPack3D: A Python Package for Online 3D Bin Packing ... <https://codeocean.com/capsule/2079012/tree>
22. Fast Collision Detection Method with Octree-Based Parallel ... - MDPI <https://www.mdpi.com/2673-4591/89/1/37>
23. [PDF] Lecture-19-BVH and Octrees <https://courses.grainger.illinois.edu/cs419/sp2017/Lecture-19-BVH%20and%20Octrees.pdf>
24. Mesh Collision Detection - Questions - three.js forum <https://discourse.threejs.org/t/mesh-collision-detection/59817>
25. Difference between BVH and Octree/K-d trees <https://computergraphics.stackexchange.com/questions/7828/difference-between-bvh-and-octree-k-d-trees>
26. [PDF] Efficient Sparse Voxel Octrees – Analysis, Extensions, and ... https://research.nvidia.com/sites/default/files/pubs/2010-02_Efficient-Sparse-Voxel-laine2010tr1_paper.pdf
27. How to improve collision detection performance on complex ... <https://stackoverflow.com/questions/45584229/how-to-improve-collision-detection-performance-on-complex-geometry-by-using-tree>
28. Frequently Asked Questions about 3DBP | 3D Bin Packing <https://www.3dbinpacking.com/en/faq>
29. Optimizing e-commerce warehousing through open dimension ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC10588690/>
30. [PDF] The Three-Dimensional Bin Packing Problem and Its Practical ... <https://scispace.com/pdf/the-three-dimensional-bin-packing-problem-and-its-practical-22gd4nfrsx.pdf>
31. 3D collision detection - MDN - Mozilla https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_collision_detection
32. useBVH (Bounding Volume Hierarchy) composable #604 - GitHub <https://github.com/Tresjs/cientos/issues/604>
33. skjolber/3d-bin-container-packing: A variant of the Largest ... - GitHub <https://github.com/skjolber/3d-bin-container-packing>
34. Where to start on encapsulating a 3D mesh in a Bounding Volume ... <https://stackoverflow.com/questions/6951636/where-to-start-on-encapsulating-a-3d-mesh-in-a-bounding-volume-hierarchy>
35. Best/Efficient way to implement a 3d Collision <https://gamedev.stackexchange.com/questions/12318/best-efficient-way-to-implement-a-3d-collision>
36. Efficient collision detection using hybrid medial axis transform and ... <https://www.sciencedirect.com/science/article/pii/S1524070323000103>
37. 3 dimensional bin packing algorithms - Stack Overflow <https://stackoverflow.com/questions/2192087/3-dimensional-bin-packing-algorithms>

38. A curated list of awesome collision detection libraries and resources <https://github.com/jslee02/awesome-collision-detection>
39. Improved Approximation Algorithms for Three-Dimensional Bin ... <https://arxiv.org/abs/2503.08863>
40. Solving a 3D bin packing problem with stacking constraints <https://www.sciencedirect.com/science/article/abs/pii/S0360835223008380>